Homework 3

Deadline: see web page
The last problem is a group assignment.
All programs have to be written in Lustre, C or python (depends on the problem) and turned in with a corresponding Makefile. They will be tested and graded on the ARC cluster, where they have to run. Familiarize yourself with this facility by following this link. If they do not compile (with the Makefile) and run, there will be no points.

1. (10 points) Determine if the task set is schedulable according to (a) deadline monotone scheduling under the system utilization test and then (b) GTDA (Generalized Timing Demand Analysis), but this time with the priority assignment in the listed order (T1,T3,T2). Also calculate the in-phase busy period. T1 = (7, 1), T2 = (12, 5), T3 = (9, 3, 11)

   Turn in file problem1.txt.

2. (20 points) Extend your prior EDF scheduling program to support static priority scheduling under Linux on a uniprocessor (single core). Create a task set and plot its execution in a Gantt chart (as static.png). Details as before, additionally:
   - Your Makefile2 should generate a binary executable file "rtsched", which accepts the name of the input file as its command argument, i.e., rtsched -static {inputFileName}. Here is a sample task set as input, numbers are in milliseconds.
   - For static priority scheduling, the input file shall list tasks in priority order (highest priority first), i.e., if you list them in increasing period, they should be scheduled in rate (or deadline) monotone priority order.

     Step-by-step inside main:

   - Initialize a thread attribute with pthread_attr_init(). Use pthread_attr_setschedpolicy() to indicate the SCHED_FIFO policy, and pthread_attr_setinheritsched with PTHREAD_EXPLICIT_SCHED. Pass the attribute at thread creating time. All else should be the same as in EDF.

     Step-by-step inside the thread:

   - For FIFO, sched_setattr() does not need to be called.

   Turn in (updated) files rtsched.c, Makefile2, static.png and README.

3. (10 points) Change your Lustre switch program into a verifier and check for correctness using Luke. Try to proof/disproof the following properties with the Luke model checker based on the "temporal" property templates of this paper:
   - ok1: After the light has been on two ticks in sequence, x becomes positive at some point.
   - ok2: After x is positive, the disjunction of light is on or x is zero holds forever.
   - ok3: After there is not light, x is zero at some point.
   - ok4: Come up with another time-based invariant of similar complexity and show that it holds.

   Document your observations and comment on correctness as well as tool limitations in README. Keep it brief.

   Turn in files verify4.lus, README (no Makefile).

4. (30 points) Enhance your Lustre program for cruise control to use a PID controller in controlSpeed() modeling with the data types

```
typedef struct _pid_params
{
// Gain parameters
    float  Kgain;        // Loop gain parameter
    float  Ti;           // Integrator time constant
    float  Td;           // Differentiator time constant
    float  delT;         // Update time interval

// Setpoint parameters
    float  setpt;        // Regulated distance to maintain
    float  input;        // Actual distance

// Limits
    int highlim;         // Upper bound
    int lowlim;          // Lower bound
} pid_params_t;

typedef struct  _pid_state
{
// Controller state
    float  proportional;
    float  integral;   // Summation of setpoint errors
    float  deriv;      // Previous setpoint error
    float  output;     //ouput of pid controller
} pid_state_t;
```

   and the corresponding controller code.

```
float seterr, change, pidout;
seterr = pid_params.setpt - pid_params.input;

// Proportional response
pidout = seterr;
pid_state->proportional = seterr;

// Intergral response
pidout += (pid_state->integral * pid_params.delT / pid_params.Ti);
pid_state->integral += seterr;

// Derivetive response
change = seterr - pid_state->deriv;
pidout += change * pid_params.Td / pid_params.delT;
pid_state->deriv = seterr;

pidout = pidout * pid_params.Kgain;

// Enforce output limits
if  (pidout > pid_params.highlim)
 pidout = pid_params.highlim;
if  (pidout < pid_params.lowlim)
 pidout = pid_params.lowlim;

pid_state->output = pidout;
```

Experimentally determine good PID coefficients and demonstrate them by comparing your old solution to your new solution. You can use command line output of graphical output from Lucille.

Turn in files cruisectr2.lus, Makefile3 with targets cruisectr2-sim and cruisectr2-exe, README3.

5. (30 points, group assignment) Given the Carla simulator and its github repo, develop your own PID-controlled autonomous driving agent, and send it (use the Jeep again) on a race drive through town with the objective to be fast instead of being safe. Tweak the existing PID controller to this end. You may use additional hard-coded tricks for driving control. See prior HW1 for CARLA setup details.
   You should start with a copy of the automatic control and add a new agent called "MyPID". Start out with the same code as for the "Basic" agent. Then make the following changes:
   - Extend the BasicAgent to take additional initialization parameters or provide new interfaces to allow a different set of PID control parameters to be relayed from my_automatic_control.py. Longitudinal control is most receptive to changes. You will have to modify the file PythonAPI/carla/agents/navigation/local_planner.py to this end as well.
   - You may disregard traffic signals and signs, and speed limits.
   - You should start at spawn location (x=-45.235935, y=-36.500095, z=0.600000) with a destination of (x=67.659737, y=69.835068, z=0.000000) in the default Town10HD.
   - You should record the start and end times as follows:

     ```
     start_time = world.hud.simulation_time #new code
     print("start time", start_time)        #new code

     clock = pygame.time.Clock()            #old code
     ...
     else: #old code, line below is modified
         print("The target has been reached, stopping the simulation, total time is", world.hud.simulation_time - start_time, "invasions", w
         break
     ```

     This requires you to also add counters to the collision and lane invasion classes, which are incremented on collision (once per colliding object) and all invasions except for carla.libcarla.LaneMarkingType.Broken (crossing broken lines is fine, but neither solid lines nor type NONE, among others). In addition, messages should be written to stdout document your trail runs.
   - You should try to obtain the shortest total time with the smallest number of penalties/collisions, each of which count for 5 seconds (except that the last two NONE penalties may be ignored).
   - Have a look at some of the other files as well, they may help:

     ```
     PythonAPI/carla/agents/navigation/controller.py #longitudinal/lateral PID control
     PythonAPI/carla/agents/navigation/base_agent.py #starting point for an agent
     ```

   Turn in "my_automatic_control.py", "local_planner.py" and README4 containing your race results. Keep your answer brief but specific. An example of the output is given below, please follow the same output formatting:

   ```
   start time 2708.4803261295892
   invasion {carla.libcarla.LaneMarkingType.Broken}
   invasion {carla.libcarla.LaneMarkingType.Broken}
   invasion {carla.libcarla.LaneMarkingType.Broken}
   invasion {carla.libcarla.LaneMarkingType.Broken}
   invasion {carla.libcarla.LaneMarkingType.NONE}
   invasion {carla.libcarla.LaneMarkingType.NONE}
   The target has been reached, stopping the simulation, total time is 23.88469760864973 invasions 2 collisions 0
   ```

What to turn in for programming assignments:

- commented program(s) as source code, comments count 15% of the points
- Makefiles (if required)
- test programs as source (and input files, if required)
- README (documentation to outline solution and list commands to install/execute)<\li>